

Desenvolvimento Web 2

Aula 4

Orientação a Objetos em JavaScript **Métodos e Propriedades dos Objetos**

`l.bertholdo@ifsp.edu.br`

Conteúdo

- Métodos de Objetos
- Objeto this
- Manipulando this
- Propriedades de Objetos
- Verificando a Existência de Propriedades
- Removendo Propriedades
- Enumeração de Propriedades
- Tipos de Propriedade
- Atributos de Propriedades
- Evitando Modificações em Objetos

Métodos de Objetos

- Em JavaScript, um objeto pode ser constituído de **propriedades de dados** e **propriedades do tipo função** (ou **métodos**, que nada mais são do que propriedades que podem ser executadas).
- Por conversão, podemos chamar “propriedades de dados” simplesmente de **propriedades** e “propriedades do tipo função” de **função**.
- No código abaixo, a variável **pessoa** recebe um objeto composto por uma **propriedade** chamada **nome** e uma **função** chamada **retornaNome()**:

```
var pessoa = {  
  nome: "Pedro",  
  retornaNome: function() {  
    console.log(pessoa.nome);  
  }  
}  
pessoa.retornaNome(); // "Pedro"
```

*A sintaxe para o valor de uma **propriedade** e de uma **função** é a mesma: um identificador seguido de dois pontos e seu respectivo valor.*

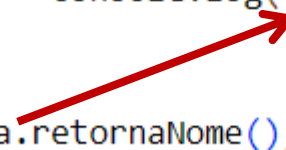
Objeto this

```
retornaNome: function() {  
    console.log(pessoa.nome);  
}
```



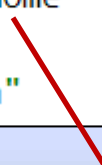
- No exemplo anterior, a função **retornaNome()** referencia a propriedade **nome** por meio do objeto **pessoa**, o que gera um alto acoplamento.
- Se o nome da variável **pessoa** for alterado, será preciso alterar a referência a ela dentro da função. Além disso, este acoplamento impede que outros objetos invoquem a função **retornaNome()**.
- Para resolver esse problema e evitar a **referência direta** a um objeto dentro de uma função, JavaScript possui a palavra reservada **this**, que representa o **objeto específico que invoca a função**:

```
var pessoa = {  
    nome: "Pedro",  
    retornaNome: function() {  
        console.log(this.nome);  
    }  
}  
pessoa.retornaNome(); // "Pedro"
```



Agora **retornaNome()** referencia **this** em vez de **pessoa**. Com isso, a função pode ser invocada por outros objetos.

```
var pessoa2 = {  
    nome: "Maria",  
    // Reutilizando a função no objeto pessoa2:  
    retorna: pessoa.retornaNome  
}  
pessoa2.retorna(); // "Maria"
```



Aqui, o objeto **pessoa2** reutiliza a função **retornaNome()** definida no objeto **pessoa**.

Objeto this

- No exemplo a seguir, é criada uma função chamada **retornaNomeGenerico()**. Em seguida, dois objetos são criados, onde **retornaNome** é definido para ser igual à função **retornaNomeGenerico()**. Por fim, é definida uma variável global chamada **nome**.

```
function retornaNomeGenerico() {  
    console.log(this.nome);  
}  
var pessoa1 = {  
    nome: "Pedro",  
    retornaNome: retornaNomeGenerico  
};  
var pessoa2 = {  
    nome: "Maria",  
    retornaNome: retornaNomeGenerico  
};  
var nome = "João";  
pessoa1.retornaNome(); // "Pedro"  
pessoa2.retornaNome(); // "Maria"  
retornaNomeGenerico(); // "João"
```

Quando a função **retornaNome()** é chamada por **pessoa1**, é exibido "Pedro". Quando é chamada por **pessoa2**, é exibido "Maria".

Quando **retornaNomeGenerico()** é chamada diretamente, o *script* considera o **this** global. Este por sua vez invoca a variável global **nome**, cujo valor é "João".

Isso ocorre porque **this** é definido no momento em que a função é chamada.

Manipulando this

- Embora **this** seja definido automaticamente, seu valor pode ser manipulado para obter resultados diferentes. Para isso, a JavaScript oferece três métodos nativos para funções: **call()**, **apply()** e **bind()**.

```
function retornaNomeGenerico(label) {  
    console.log(label + ":" + this.nome);  
}  
  
var pessoa1 = {  
    nome: "Pedro"  
};  
  
var pessoa2 = {  
    nome: "Maria"  
};  
  
var nome = "João";  
  
retornaNomeGenerico.call(this, "global");    // global:João  
retornaNomeGenerico.call(pessoa1, "pessoa1"); // pessoa1:Pedro  
retornaNomeGenerico.call(pessoa2, "pessoa2"); // pessoa2:Maria
```

O método **call()** executa a função com um determinado valor de **this**. O 1º parâmetro é o valor que **this** deve ter quando a função for executada. Os parâmetros seguintes são aqueles esperados pela função que invoca o método **call()**, no caso, a função **retornaNomeGenerico(label)**.

Nesse exemplo, a 1ª chamada da função passa o **this** global e o parâmetro "global" como label. Na 2ª chamada, a função define explicitamente o objeto **pessoa1** como **this** e passa o parâmetro "pessoa1" como label. Na 3ª chamada, o mesmo é feito para o objeto **pessoa2**. Note que a função é usada como um objeto para chamar **call()**.

Manipulando this

```
function retornaNomeGenerico(label) {  
    console.log(label + ":" + this.nome);  
}  
  
var pessoa1 = {  
    nome: "Pedro"  
};  
  
var pessoa2 = {  
    nome: "Maria"  
}  
  
var nome = "João";  
retornaNomeGenerico.apply(this, ["global"]);    // global:João  
retornaNomeGenerico.apply(pessoa1, ["pessoa1"]); // pessoa1:Pedro  
retornaNomeGenerico.apply(pessoa2, ["pessoa2"]); // pessoa2:Maria
```

O método **apply()** funciona exatamente como **call()**, exceto pelo fato de ele receber somente dois parâmetros: o valor de **this** e um *array* (ou objeto similar) contendo os parâmetros esperados pela função chamada.

Manipulando this

```
function retornaNomeGenerico(label) {  
    console.log(label + ":" + this.nome);  
}  
  
var pessoa1 = {  
    nome: "Pedro"  
};  
  
var pessoa2 = {  
    nome: "Maria"  
}  
  
// Cria uma função somente para "pessoa1"  
var retornaNomePessoa1 = retornaNomeGenerico.bind(pessoa1);  
retornaNomePessoa1("pessoa1"); // pessoa1:Pedro  
  
// Cria uma função somente para "pessoa2"  
var retornaNomePessoa2 = retornaNomeGenerico.bind(pessoa2, "pessoa2");  
retornaNomePessoa2();           // pessoa2:Maria
```

O método **bind()** se comporta de modo diferente dos dois anteriores. O 1º parâmetro é o valor de **this** para uma função a ser criada. Os parâmetros seguintes são aqueles esperados pela função chamada, porém são **opcionais**.

Em **retornaNomePessoa1**, **bind** vincula o valor de **this** à **pessoa1**, sem passar o parâmetro **label** (ele é passado ao chamar o método **retornaNomePessoa1**). Em **retornaNomePessoa2**, **bind** vincula **this** à **pessoa2**, e ele mesmo passa o parâmetro **label** ("pessoa2").

A maneira como **this** se vincula ao escopo foi modificada na **ES6**. Consequentemente, os métodos **call()**, **apply()** e **bind()** não funcionam com as **arrow functions** introduzidas nessa versão.

Propriedades de Objetos

- Enquanto as linguagens baseadas em classes restringem os objetos de acordo com uma definição de **classe**, em JavaScript os objetos são **dinâmicos** e podem ser modificados durante a execução do código.
- Como vimos, há duas formas de criar seus próprios objetos: criando um **objeto na forma literal** ou usando o construtor **Object**:

```
var pessoa1 = {  
  nome: "Pedro"  
};  
var pessoa2 = new Object();  
pessoa2.nome = "Pedro";  
  
pessoa1.idade = 22;
```

*Nesse exemplo, tanto **pessoa1** quanto **pessoa2** são objetos que têm uma propriedade **nome**. Esses objetos podem receber outras propriedades ao longo do código. A menos que seja especificado o contrário, os objetos criados pelo desenvolvedor, estarão sempre abertos a modificações, como a adição ou exclusão de propriedades.*

Para cada propriedade adicionada a um objeto, é criada uma **propriedade própria** nele, ou seja, a propriedade é armazenada diretamente na instância do objeto e todas as operações sobre esta propriedade devem ser executadas por meio desse objeto.

Verificando a Existência de Propriedades

- Como as propriedades podem ser adicionadas a qualquer momento, às vezes é necessário verificar se uma propriedade já existe em um objeto. Uma maneira de fazer essa verificação é por meio do operador **in**.

```
var pessoa = {  
  nome: "Pedro"  
};  
pessoa.idade = 22;  
  
console.log("nome" in pessoa); // true  
console.log("idade" in pessoa); // true  
console.log("altura" in pessoa); // false
```

O operador **in** procura uma propriedade com um determinado nome em um objeto específico e retorna **true** se ela for encontrada.

```
var pessoa = {  
  nome: "Pedro",  
  retornaNome: function() {  
    console.log(this.nome);  
  }  
};  
console.log("retornaNome" in pessoa); // true
```

Funções também são **propriedades**, logo é possível verificar a existência delas da mesma forma.

Verificando a Existência de Propriedades

- O operador **in** verifica qualquer tipo de propriedade, inclusive propriedades herdadas. Porém, em alguns casos, pode ser necessário verificar apenas a existência de uma **propriedade própria**.
- Para verificar apenas **propriedades próprias**, é preciso usar o método **hasOwnProperty()**, presente em todos os objetos. Ele retorna **true** somente se a propriedade existir e for própria.

```
var pessoa = {  
  nome: "Pedro",  
  retornaNome: function() {  
    console.log(this.nome);  
  }  
};  
console.log("nome" in pessoa);           // true  
console.log(pessoa.hasOwnProperty("nome")); // true  
console.log("toString" in pessoa);       // true  
console.log(pessoa.hasOwnProperty("toString")); // false
```

Aqui, as duas linhas de código retornam **true**, porque **nome** existe e é uma propriedade própria.

O método **toString()**, entretanto, é uma propriedade presente em qualquer objeto. Nesse caso, o operador **in** retorna **true** e o método **hasOwnProperty()** retorna **false**.

Removendo Propriedades

- Assim como as propriedades podem ser adicionadas aos objetos a qualquer momento, elas também podem ser removidas.
- Para isso, é preciso usar o operador **delete**:

Se o operador **delete** conseguir remover a propriedade, ele retorna **true**. Esse retorno booleano faz sentido, pois nem todas as propriedades podem ser removidas.

```
var pessoa = {  
    nome: "Pedro"  
};  
console.log("nome" in pessoa); // true  
delete pessoa.nome;  
console.log("nome" in pessoa); // false  
console.log(pessoa.nome);      // undefined
```

Enumeração de Propriedades

- Por padrão, todas as propriedades adicionadas a um objeto são **enumeráveis**, o que possibilita iterá-las usando uma estrutura **for-in**.
- O *loop for-in* percorre todas as propriedades de um objeto, permitindo exibir seus nomes e seus valores.

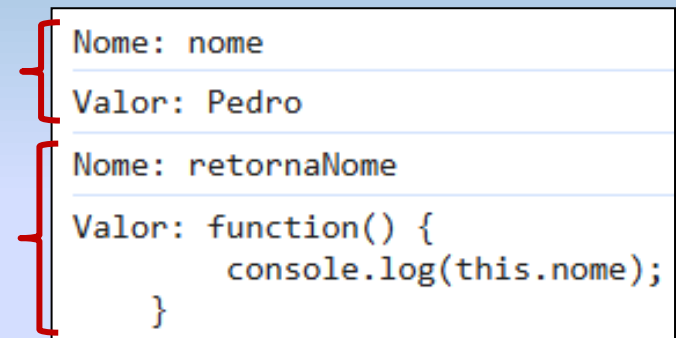
```
var pessoa = {  
  nome: "Pedro",  
  retornaNome: function() {  
    console.log(this.nome);  
  }  
};  
for (var propriedade in pessoa) {  
  console.log("Nome: " + propriedade);  
  console.log("Valor: " + pessoa[propriedade]);  
}
```

{	Nome: nome
	Valor: Pedro
{	Nome: retornaNome
	Valor: function() { console.log(this.nome); }

Enumeração de Propriedades

- Outra forma de percorrer as propriedades de um objeto é por meio do método **Object.keys()**, que retorna um *array* contendo suas propriedades.

```
var pessoa = {  
  nome: "Pedro",  
  retornaNome: function() {  
    console.log(this.nome);  
  }  
};  
var propriedades = Object.keys(pessoa);  
for (var i = 0; i < propriedades.length; i++) {  
  console.log("Nome: " + propriedades[i]);  
  console.log("Valor: " + pessoa[propiedades[i]]);  
}
```



Nome: nome
Valor: Pedro
Nome: retornaNome
Valor: function() { console.log(this.nome); }

```
console.log("nome" in pessoa); // true  
console.log(pessoa.propertyIsEnumerable("nome")); // true  
var props = Object.keys(pessoa);  
console.log("length" in props); // true  
console.log(props.propertyIsEnumerable("length")); // false  
// "length" não é uma propriedade enumerável do array "props".
```

*Nem todas as propriedades de um objeto são enumeráveis. Para verificar essa característica, pode-se usar o método **propertyIsEnumerable()**, que está presente em todos os objetos.*

Tipos de Propriedade

- Com relação aos **dados** dos objetos, existem dois tipos de propriedade:
 - As **propriedades de dados**, que contêm um valor, como a propriedade **nome** dos exemplos anteriores.
 - As **propriedades de acesso**, que não contêm valores. Em vez disso, elas definem uma função a ser chamada quando a propriedade é retornada (**get**) e outra função a ser chamada quando a propriedade é atualizada (**set**).

```
var pessoa = {  
  _nome: "Pedro",  
  get nome() {  
    console.log("Lendo nome");  
    return this._nome;  
  },  
  set nome(valor) {  
    console.log("Atualizando nome para %s", valor);  
    this._nome = valor;  
  }  
};  
  
console.log(pessoa.nome); // "Lendo nome" e, em seguida, "Pedro"  
pessoa.nome = "Maria";    // "Atualizando nome para Maria"  
console.log(pessoa.nome); // "Lendo nome" e, em seguida, "Maria"
```

*Esse exemplo define uma **propriedade de acesso** chamada **nome**. Também é criada uma **propriedade de dado** chamada **_nome**, que contém o valor da propriedade em si. O **underline** é uma convenção para indicar que a propriedade é privada, embora, nesse caso, ela continue sendo pública. As palavras-chave **get** e **set** são usadas antes do nome da propriedade de acesso, seguidas por parênteses e do corpo de uma função.*

Lendo nome
Pedro
Atualizando nome para Maria
Lendo nome
Maria

Tipos de Propriedade

- As propriedades de acesso são úteis quando se deseja que a **atualização** de um dado acione algum tipo de comportamento, ou quando o **retorno** de um dado exigir alguma condição ou cálculo do valor a ser retornado.

```
var pessoa = {  
  _nome: "Pedro",  
  get nome() {  
    console.log("Lendo nome");  
    return this._nome;  
  },  
  set nome(valor) {  
    console.log("Atualizando nome para %s", valor);  
    this._nome = valor;  
  }  
};  
  
console.log(pessoa.nome); // "Lendo nome" e, em seguida, "Pedro"  
pessoa.nome = "Maria";    // "Atualizando nome para Maria"  
console.log(pessoa.nome); // "Lendo nome e, em seguida, "Maria"
```

*As propriedades de acesso exigem apenas um **get** ou um **set**, embora ambos possam existir. Se apenas um **get** for definido, a propriedade será **somente de leitura** e uma tentativa de escrever nessa propriedade irá falhar silenciosamente. Se somente um **set** for definido, a propriedade será **somente de escrita**, e uma tentativa de retornar o valor irá falhar silenciosamente.*

Atributos de Propriedades

- Existem dois **atributos internos** que são compartilhados entre as **propriedades de dados** e as **propriedades de acesso**.
- Um deles é o **enumerable**, que determina se a propriedade pode ser iterada. O outro é **configurable**, que determina se uma propriedade pode ser alterada. Por padrão, todas as propriedades declaradas em um objeto são enumeráveis e configuráveis.

Uma **propriedade configurável** pode ser removida usando **delete** e seus atributos podem ser alterados, inclusive para mudarem de **propriedades de dados** para **propriedades de acesso**, e vice-versa.

```
var pessoa = {  
  nome: "Pedro"  
};  
Object.defineProperty(pessoa, "nome", {  
  enumerable: false  
});
```

Para mudar os atributos das propriedades, existe o método **Object.defineProperty()**, que recebe três argumentos: **1)** o objeto que tem a propriedade, **2)** o nome da propriedade e **3)** um objeto descritor da propriedade (que contém os atributos a serem definidos). Nesse exemplo, está sendo definido que a propriedade **nome** do objeto **pessoa1** não deve ser iterável.

Atributos de Propriedades

```
var pessoa = {  
  nome: "Pedro"  
};  
Object.defineProperty(pessoa, "nome", {  
  enumerable: false  
});  
console.log(pessoa.propertyIsEnumerable("nome")); // false  
  
Object.defineProperty(pessoa, "nome", {  
  configurable: false  
});  
delete pessoa.nome; // Falha ao tentar apagar a propriedade  
  
Object.defineProperty(pessoa, "nome", { // Erro!  
  configurable: true  
});
```

Após alterar o atributo **enumerable** para **false**, o método **propertyIsEnumerable()** retorna que a propriedade não é iterável.

Agora, **nome** é alterado para que se torne não configurável. A partir daí, uma tentativa de apagar **nome** falha silenciosamente, porque a propriedade não pode mais ser alterada.

Por fim, o código tenta redefinir a propriedade **nome** para que seja novamente configurável. No entanto, isso gera um erro, pois uma propriedade não configurável não pode se tornar configurável novamente.

false

✖ ▶ Uncaught TypeError: Cannot redefine property: nome
at Object.defineProperty (<anonymous>)

Atributos de Propriedades

- Para acessar os **atributos** das propriedades, existe o método **Object.getOwnPropertyDescriptor()**, que funciona apenas com **propriedades próprias**.

Este método recebe dois argumentos: o objeto a ser manipulado e o nome da propriedade a ser acessada. Se a propriedade existir, o método retornará um **objeto descritor** com os atributos **configurable**, **enumerable** e os outros atributos adequados ao tipo da propriedade.

```
var pessoa = {  
  nome: "Pedro"  
}  
var descriptor = Object.getOwnPropertyDescriptor(pessoa, "nome");  
console.log(descriptor.enumerable); // true  
console.log(descriptor.configurable); // true  
console.log(descriptor.writable); // true  
console.log(descriptor.value); // "Pedro"
```

Evitando Modificações em Objetos

- Os objetos, assim como as propriedades, têm atributos internos que definem seu comportamento. Um desses atributos é o **extensible**, que indica se o objeto permite a adição de propriedades.
- Por padrão, todos os objetos criados são **extensíveis**, o que significa que novas propriedades podem ser adicionadas ao objeto.
- Para evitar que propriedades sejam adicionadas a um objeto, existem três maneiras: 1) evitando extensões, 2) selando o objeto e 3) congelando o objeto.

```
var pessoa = {  
  nome: "Pedro"  
}  
console.log(Object.isExtensible(pessoa)); // true  
Object.preventExtensions(pessoa);  
console.log(Object.isExtensible(pessoa)); // false  
pessoa.retornaNome = function() {  
  console.log(this.nome);  
};  
console.log("retornaNome" in pessoa); // false
```

O valor do atributo **extensible** pode ser verificado com o método **Object.isExtensible()**. Para **evitar extensões em objetos**, existe o método **Object.preventExtensions()**. Esse método recebe como argumento o objeto que se deseja tornar não extensível.

Ao tentar adicionar uma propriedade a um objeto não extensível, a operação falhará silenciosamente.

Evitando Modificações em Objetos

- A segunda forma de criar um objeto não extensível é **selando o objeto**.
- Um **objeto selado** é não extensível, e todas as suas propriedades são não configuráveis, ou seja, não é possível adicionar, remover nem mudar o tipo das propriedades (de dados para acesso, ou vice-versa). É possível apenas **ler** e **atualizar** os valores das propriedades do objeto.

```
var pessoa = {  
  nome: "Pedro"  
}  
console.log(Object.isSealed(pessoa)); // false  
Object.seal(pessoa);  
console.log(Object.isSealed(pessoa)); // true  
  
pessoa.idade = 25;  
console.log(pessoa.idade); // undefined  
  
delete pessoa.nome;  
console.log("nome" in pessoa); // true  
  
pessoa.nome = "Maria";  
console.log(pessoa.nome); // "Maria"
```

Para verificar se um objeto está selado, usa-se o método **Object.isSealed()**. Já para **selar um objeto**, existe o método **Object.seal()**, que define os atributos **extensible** (do objeto) e **configurable** (das propriedades) como **false**.

Esse código **sela** o objeto **pessoa**, de modo que ao tentar **adicionar** ou **remover** propriedades, a operação falha silenciosamente.

No entanto, ainda é possível **atualizar** e **ler** os valores de suas propriedades.

Evitando Modificações em Objetos

- A última forma de criar um objeto não extensível é **congelando o objeto**.
- Um **objeto congelado** não permite adicionar, remover, mudar o tipo ou atualizar o valor de suas propriedades. É possível apenas **ler** os valores das propriedades do objeto.

```
var pessoa = {  
  nome: "Pedro"  
}  
  
console.log(Object.isFrozen(pessoa)); // false  
Object.freeze(pessoa);  
console.log(Object.isFrozen(pessoa)); // true  
  
pessoa.idade = 25;  
console.log(pessoa.idade); // undefined  
  
delete pessoa.nome;  
console.log("nome" in pessoa); // true  
  
pessoa.nome = "Maria";  
console.log(pessoa.nome); // "Pedro"
```

*Para verificar se um objeto está congelado, usa-se o método **Object.isFrozen()**. Já para **congelar um objeto**, existe o método **Object.freeze()**.*

Objetos congelados não podem ser “descongelados”, permanecendo no estado em que estavam quando foram congelados.

Esse código **congela** o objeto **pessoa**, de modo que ao tentar **adicionar** ou **remover** propriedades, a operação falha silenciosamente.

Além disso, não é possível **atualizar** os valores de suas propriedades.

Referências

- ZAKAS, Nicholas C. **Princípios de Orientação a Objetos em JavaScript**. São Paulo: Editora Novatec, 2014.